

Sprint Documentation

Sprint 15	Start Date: 14/04/2026	Final Date: 20/04/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Wendell Alves Nascimento		
Gustavo Henrique Moreira Porto		

Sprint Backlog

Task#	Description	Start Date	Final date
#78	Validações de Instituição	14/04/2026	20/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#78	Garantir que o cadastro e manutenção de instituições possuam validações completas de integridade, padronização e regras de negócio, evitando dados inconsistentes e garantindo confiabilidade no sistema.	Wendell Alves Nascimento, Gustavo Henrique Moreira Porto	Completo	7 horas	7 horas

Sprint Description

Tarefa: Validações de Instituição

Objetivo

Garantir que o cadastro e manutenção de instituições possuam validações completas de integridade, padronização e regras de negócio, evitando dados inconsistentes e garantindo confiabilidade no sistema.

Descrição

Implementar validações no backend para os campos da entidade de instituição, assegurando:

- Dados obrigatórios

- Formatação correta
- Normalização de dados
- Unicidade no banco
- Integridade relacional

Critérios de Aceite

◆ Campos Obrigatórios

- Validar que os campos obrigatórios não estão vazios:
 - cnpj
 - nome
 - logradouro
 - numero
 - bairro
 - cep
 - nomeRazaoSocial
 - telefone
 - email
 - cidade
 - estado
 - situacao
 - idTipoInstituicao
 - idSecretaria

◆ Normalização de Dados

- Permitir entrada com máscara para:
 - CNPJ
 - CEP
 - Telefone
- Remover caracteres não numéricos antes de processar:
 - CNPJ
 - CEP
 - Telefone

Armazenar apenas números no banco (quando aplicável)

- Padronizar:

- Email em minúsculo
- Estado em maiúsculo (UF)

◆ Validação de CNPJ

- Deve conter 14 dígitos após normalização

Deve ser um CNPJ válido

Não permitir CNPJ duplicado no sistema

Rejeitar CNPJs inválidos

◆ Validação de Nome

- Mínimo de 3 caracteres

Máximo de 150 caracteres

◆ Validação de Razão Social

- Campo obrigatório

Mínimo de 3 caracteres

Máximo de 200 caracteres

◆ Validação de Email

- Validar formato de email válido

Máximo de 255 caracteres

Não permitir duplicidade

◆ Validação de Telefone

- Deve conter entre 10 e 11 dígitos após normalização

Permitir apenas números

Validar formato básico (DDD + número)

◆ Validação de Endereço

- logradouro: obrigatório (máx: 255)

numero: obrigatório

bairro: obrigatório

cidade: obrigatório

- estado:

- Deve conter uma UF válida (ex: SP, RJ, MG)
- cep:
 - Deve conter 8 dígitos após normalização

◆ Validação de Situação

- Deve corresponder a um valor válido do enum (Situacao)
- Exemplo:
 - 1 = Ativo
 - 2 = Inativo

Rejeitar valores fora do padrão

◆ Regra de Negócio (Situação)

-
- Instituições INATIVAS:
 - Não podem receber novos orçamentos
 - Não podem lançar novas despesas

◆ Validação de Tipo de Instituição

- idTipoInstituicao deve existir no banco

Validar integridade da chave estrangeira

◆ Validação de Secretaria

- idSecretaria deve existir no banco

Validar integridade da chave estrangeira

Garantir que a instituição esteja vinculada a uma secretaria válida

◆ Regra de Unicidade

-
- Não permitir duplicidade de:
 - CNPJ (obrigatório)

◆ Validação em Atualização

-
- Ao alterar situacao para INATIVO:

- Validar impacto em orçamentos e despesas

Garantir consistência dos dados após alteração

◆ Normalização e Boas Práticas

- Validar dados antes de persistir no banco

Implementar validações na camada **SERVICE**

Não depender apenas de **DataAnnotations**

Retornar mensagens claras ao usuário

Observações e Sugestões

- Aplicar validações consistentes entre backend e banco
- Utilizar transações em operações críticas
- Manter código reutilizável e escalável
- Os erros retornados ao usuário devem vir em uma lista com todos os erros de uma só vez ao invés de mostrar um erro por vez.
- Verifiquem o trabalho antes do envio

Sprint Results

Arquivos CRIADOS (novos)

Civitas.WebAPI/Services/Validation/InstituicaoValidationException.cs

Exception para erros de validação com coleção IReadOnlyCollection<string> Errors

Padrão copiado de UsuarioValidationException

Civitas.WebAPI/Services/Validation/InstituicaoConflictException.cs

Exception para conflitos de unicidade com propriedade string Field

Padrão copiado de UsuarioConflictException

Civitas.WebAPI.Tests/InstituicaoValidationEndpointsTests.cs (+445 linhas)

9 testes de integração cobrindo: normalização de campos mascarados, CNPJ inválido, CNPJ duplicado (409), email duplicado (409), FK inválida, múltiplos erros simultaneamente, update válido, PATCH com despesas pendentes, despesa com instituição inativa

Arquivos MODIFICADOS

InstituicaoService.cs (+435 linhas, -75)

Adicionadas 2 dependências no construtor: ITipoInstituicaoRepository, ISecretariaRepository

Adicionados campos estáticos: EmailValidator e ValidUfs (HashSet com as 27 UFs)

Override Create (novo): ValidateDtoInstance → Normalize → ValidateFields → ValidateRelationships → ValidateUniqueness → salva via repository

Override Update (novo): mesma pipeline + ValidateStatusTransitionAsync + salva via repository diretamente (sem chamar base.Update para evitar double GetById)

Normalize: normaliza CNPJ/CEP/Telefone (só dígitos), textos (Trim), Estado (ToUpperInvariant), Email (ToLowerInvariant)

ValidateFields: coleta TODOS os erros em List<string> antes de lançar exception — obrigatoriedade, tamanhos (Nome 3-150, RazaoSocial 3-200, Email/Logradouro max 255), formato (CNPJ mod11, Email, Telefone 10-11 dígitos, CEP 8 dígitos, Estado UF válida), enum Situacao

ValidateRelationships: verifica se TipoInstituicao e Secretaria existem no banco

ValidateStatusTransitionAsync: bloqueia transição ATIVO→INATIVO se houver despesas pendentes

ValidateUniqueness: bloqueia CNPJ duplicado e Email duplicado

IsCnpj: algoritmo módulo-11 (copiado de FornecedorService)

OnlyDigits: extrai somente dígitos de uma string

IIstituicaoRepository.cs (+3 linhas)

Task<bool> ExistsByCnpjAsync(string cnpj, int? ignoreId = null)

Task<bool> ExistsByEmailAsync(string email, int? ignoreId = null)

Task<bool> HasDespesasPendentesAsync(int instituicaoId)

InstituicaoRepository.cs (+45 linhas)

Implementações dos 3 métodos acima com AsNoTracking() e suporte a ignoreId

HasDespesasPendentesAsync: AnyAsync(d => d.IdInstituicao == id && d.Status == Status.A_PAGAR)

InstituicaoBuilder.cs (+14 linhas)

Nome: MaxLength 100 → 150

NomeRazaoSocial: MaxLength 100 → 200

Email: MaxLength 100 → 255

Logradouro: MaxLength 200 → 255

Estado: MaxLength 100 → 2

InstituicaoController.cs (+296 linhas, -256 removidas/reescritas)

POST: adicionados catch para InstituicaoConflictException (409) e InstituicaoValidationException (400)

PUT: removido check de existência do controller (agora no service); adicionados catch para KeyNotFoundException (404) e as novas exceptions

PATCH: adicionados catch blocks; a validação de despesas pendentes ocorre via Update → ValidateStatusTransitionAsync

DespesaService.cs (+5 linhas)

Linhas 101-104: após verificar existência da instituição, adicionado:

```
if (instituicao.Situacao != Situacao.ATIVO)
```

```
    throw new ArgumentException("Instituicao inativa nao pode receber novas despesas.");
```

FornecedorValidationEndpointsTests.cs (+4 linhas)

Adicionado seed de TipoCodigo que estava faltando como dependência do TipoDespesa

Migration gerada automaticamente

Civitas.WebAPI/Migrations/20260415194201_AtualizarConstraintsInstituicao.cs

Altera as 5 colunas acima com Up/Down reversível

Resultado final

Build: 0 erros

Testes: 43 passaram (incluindo os 9 novos), 4 falhas pré-existent não relacionadas

Assinaturas